PYTHON FOR DATA · 35 MIN

NumPy Basics: Fast Element-wise Functions

Arrays
One type, one block
`np.array`

Vectorised
No Python loops
`a * 2`

Ufuncs
Functions per element
`np.sqrt`

Broadcasting
Mix array shapes
`a + row`

~3 min

Four Ideas Behind Fast Arrays

Work on whole arrays instead of one value at a time.

1

Arrays

A grid of numbers that share one data type.

np.array

2

Vectorising

An operation applies to every element at once.

a * 2

3

Ufuncs

Universal functions work element by element.

np.sqrt

4

Broadcasting

Arrays of different shapes can still combine.

a + row

The idea: write the operation once, and let NumPy run the loop in fast compiled code.

~4 min

The ndarray

One block of same-typed values.

```
import numpy as np
a = np.array([1, 2, 3])
print(a.dtype, a.shape)
print(a * 2)
```

Output

```
int64 (3,)
[2 4 6]
```

ndarray

A grid of values that all share one dtype, stored together in one block of memory.

Why it is fast

Operations run in compiled C loops over that block, not in Python bytecode.

~4 min

No Loops Needed

The same job, with and without NumPy.

```
nums = [1, 2, 3]
print([n * 2 for n in nums])
a = np.array(nums)
print(a * 2)
```

Output

```
[2, 4, 6]
[2 4 6]
```

Vectorisation

Write the operation once. NumPy applies it to every element.

Lists behave differently

nums * 2 on a list repeats it. On an array it multiplies each number.

~4 min

Element-wise Arithmetic

Operators work position by position.

Expression	Result
a + b	[11 22 33]
a * b	[10 40 90]
b / a	[10. 10. 10.]
a ** 2	[1 4 9]
a > 1	[False True True]

Here a = np.array([1, 2, 3]) and b = np.array([10, 20, 30]). Both need the same shape, or shapes that broadcast.

~4 min

Unary Universal Functions

One array in, one array out.

```
a = np.array([1, 4, 9])
print(np.sqrt(a))
print(np.abs(np.array([-2, 3])))
print(np.square(a))
```

Output

```
[1. 2. 3.]
[2 3]
[ 1 16 81]
```

Unary ufuncs

Take one array and return a new array of the same shape, such as sqrt, exp, log, abs and sin.

A new array each time

The original array is left unchanged unless you pass out= to store the result.

~4 min

Common Universal Functions

Fast functions that act on every element.

Function	What it does	Kind
np.sqrt	Square root of each element	Unary
np.exp	e raised to each element	Unary
np.abs	Absolute value of each element	Unary
np.add	Element-wise a + b	Binary
np.maximum	Larger of each pair	Binary

np.add(a, b) and a + b do the same thing. The operators are shortcuts for ufuncs.

~4 min

Binary Functions and where

Combine two arrays, or choose by condition.

```
x = np.array([1, 5, 3])
y = np.array([4, 2, 6])
print(np.maximum(x, y))
print(np.add(x, y))
print(np.where(x > 2, 1, 0))
```

Output

```
[4 5 6]
[5 7 9]
[0 1 1]
```

Binary ufuncs

Take two arrays: add, subtract, multiply, maximum, minimum, mod and more.

np.where

A vectorised if-else. It picks one value or another for each element, based on a condition.

~5 min

Broadcasting

Mix arrays of different shapes.

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
print(a * 10)
print(a + np.array([100, 200, 300]))
```

Output

```
[[10 20 30]
 [40 50 60]]
[[101 202 303]
 [104 205 306]]
```

Broadcasting

NumPy stretches a smaller array across a bigger one when their shapes are compatible.

Rule of thumb

Shapes are compared from the right. Each pair of sizes must match, or one of them must be 1.

~4 min

Summaries Along an Axis

Collapse an array to fewer values.

```
a = np.array([[1, 2], [3, 4]])
print(a.sum(), a.mean())
print(a.sum(axis=0))
print(a.max(axis=1))
```

Output

```
10 2.5
[4 6]
[2 4]
```

Reductions

sum, mean, min and max collapse an array to one value, or to one value per row or column.

axis

axis=0 works down the columns. axis=1 works across the rows.

~3 min

Why Vectorising Wins

Where the time goes in each approach.

Approach	How it runs	Speed
Python for loop	One element at a time, in Python	Slow
List comprehension	Same loop, shorter code	Slow
NumPy ufunc	A compiled C loop over one memory block	Fast

> On large arrays a vectorised version is often many times faster than the equivalent Python loop.

ONE-LINE TAKEAWAY

Write the operation once, and NumPy applies it to every element in fast compiled code.

ndarray ufunc broadcasting axis

PYTHON FOR DATA · 40 MIN

Multidimensional Arrays

Shape
Rows and columns
`a.shape`

Indexing
Pick one item
`a[1, 2]`

Slicing
Pick a region
`a[:, 0]`

Reshaping
Rearrange data
`a.reshape`

~3 min

Four Skills for Working in Many Dimensions

From building an array to rearranging it.

1

Creating

Nested lists become arrays with any number of axes.

np.array

2

Shape and axes

Know how many items lie along each direction.

shape ndim

3

Indexing

Reach one item, one row or a whole region.

a[1, 2]

4

Reshaping

Change the layout and join arrays together.

reshape

A multidimensional array is still one block of memory, viewed through a shape.

~4 min

Creating Arrays

Nested lists make a grid of values.

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
print(a.ndim, a.shape, a.size)
print(np.zeros((2, 2)))
```

Output

```
2 (2, 3) 6
[[0. 0.]
 [0. 0.]]
```

Nested lists

A list of equal-length lists makes a 2-D array. Add another level of nesting for 3-D.

Built-in creators

zeros, ones, empty, arange and full build arrays from a shape, without typing every value.

~4 min

Array Attributes

What every array can tell you about itself.

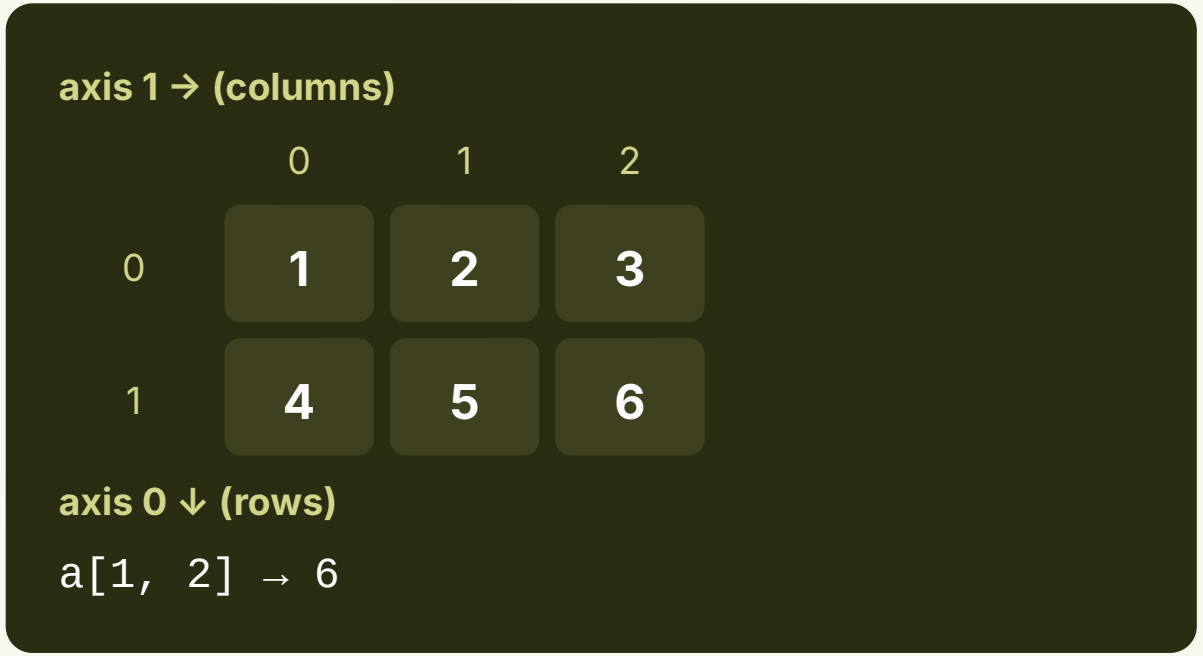
Attribute	Meaning	For the array above
ndim	Number of dimensions	2
shape	Size along each axis	(2, 3)
size	Total number of elements	6
dtype	Type of each element	int64
T	Transposed view	shape (3, 2)

The array above is [[1, 2, 3], [4, 5, 6]]. The dtype shown is typical, and can differ by platform.

~4 min

Rows, Columns and Axes

Each direction in an array is an axis.



Axis 0

Runs down the rows. a.sum(axis=0) gives one total per column.

Axis 1

Runs across the columns. a.sum(axis=1) gives one total per row.

~4 min

Indexing

Reach one item or one row.

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
print(a[1, 2])
print(a[0])
print(a[1][0])
```

Output

```
6
[1 2 3]
4
```

a[row, col]

Put the indexes in one bracket, separated by commas. a[1, 2] is row 1, column 2.

Fewer indexes

a[0] gives the whole first row. Leaving out an axis returns a smaller array.

~4 min

Slicing

Pick a region with start:stop:step.

```
a = np.arange(12).reshape(3, 4)
print(a[:, 1])
print(a[1:, :2])
```

Output

```
[1 5 9]
[[4 5]
 [8 9]]
```

One slice per axis

Each axis gets its own slice, separated by commas. A lone colon means the whole axis.

Views, not copies

A slice shows the same data. Changing it also changes the original array.

~4 min

Boolean and Fancy Indexing

Choose elements by condition or by list.

```
a = np.array([[1, 2], [3, 4], [5, 6]])
print(a[a > 3])
print(a[[0, 2]])
```

Output

```
[4 5 6]
[[1 2]
 [5 6]]
```

Boolean mask

A True/False array of the same shape picks matching elements, returned as a flat array.

Fancy indexing

A list of row numbers picks those rows in that order. Both forms return copies, not views.

~4 min

Reshaping and Transposing

Same data, a different layout.

```
a = np.arange(6)
b = a.reshape(2, 3)
print(b.T)
```

Output

```
[[0 3]
 [1 4]
 [2 5]]
```

reshape

Changes the shape without changing the data. The total number of elements must stay the same.

-1, flatten and T

reshape(-1) lets NumPy work out one size. flatten() returns a 1-D copy. T swaps the axes.

~4 min

Joining Arrays

Stack arrays top to bottom or side by side.

```
a = np.array([[1, 2]])
b = np.array([[3, 4]])
print(np.vstack([a, b]))
print(np.hstack([a, b]))
```

Output

```
[[1 2]
 [3 4]]
[[1 2 3 4]]
```

Joining arrays

vstack stacks on top, and hstack places side by side. concatenate takes an axis= argument.

Matching shapes

Sizes along every other axis must agree, or NumPy raises a ValueError.

~4 min

Beyond Two Dimensions

Axes keep adding on.

```
c = np.zeros((2, 3, 4))
print(c.shape, c.ndim)
print(c[1].shape)
print(c[:, 0, :].shape)
```

Output

```
(2, 3, 4) 3
(3, 4)
(2, 4)
```

A stack of tables

Read a 3-D array as shape (stacks, rows, columns).

Pick one, lose one

An integer index removes that axis. A slice keeps it.

~3 min

Shape Tools at a Glance

Match the task to the function.

Task	Tool	Result
Change the shape	reshape	Same data, new shape
Swap axes	T or transpose	Rows become columns
Join arrays	vstack, hstack, concatenate	One bigger array
Split an array	np.split	A list of smaller arrays
Make it 1-D	ravel or flatten	A flat array

ravel returns a view when it can, while flatten always returns a copy.

ONE-LINE TAKEAWAY

Shape and axes describe an array, and indexing, slicing and reshaping let you reach and rearrange any part of it.

shape axis slice reshape

PYTHON FOR DATA · 45 MIN

Data Processing Using Arrays

Vectorising
Whole-array maths
`xs ** 2`

Conditions
Array if-else
`np.where`

Statistics
Summaries of data
`mean std`

Organising
Sort and find sets
`sort unique`

~3 min

Four Steps in Array Data Processing

Whole-array thinking from raw numbers to saved results.

1

Vectorise

Replace loops with whole-array expressions.

a ** 2

2

Decide

Express if-else logic as array operations.

np.where

3

Summarise

Statistics and true or false checks on data.

mean any

4

Organise

Sort, find unique values, save and load.

sort unique

Array-oriented code is shorter than loops, and runs in fast compiled code.

~5 min

Evaluating a Formula on a Grid

One expression, every (x, y) pair.

```
pts = np.arange(-1, 2)
xs, ys = np.meshgrid(pts, pts)
z = np.sqrt(xs ** 2 + ys ** 2)
print(z.round(1))
```

Output

```
[[1.4 1. 1.4]
 [1. 0. 1. ]
 [1.4 1. 1.4]]
```

meshgrid

Turns two 1-D arrays into two 2-D grids, so a formula can be evaluated at every (x, y) pair.

Array-oriented code

Replacing loops with array expressions is called vectorisation. It is usually shorter and faster.

~4 min

Conditional Logic

if-else, applied to a whole array.

```
temp = np.array([18, 25, 31, 22])
print(np.where(temp > 24, "hot", "mild"))
print(np.where(temp > 24, temp, 0))
```

Output

```
['mild' 'hot' 'hot' 'mild']
[ 0 25 31 0]
```

np.where(cond, a, b)

A vectorised if-else. It takes a where the condition is True, and b everywhere else.

Faster than a loop

The test and the choice happen for the whole array in one step.

~4 min

Statistical Methods

Summaries that work on a whole array.

Call	Meaning	Result
a.sum()	Total of all elements	10
a.mean()	Average	2.5
a.std()	Standard deviation	1.118
a.cumsum()	Running totals	[1 3 6 10]
a.argmax()	Position of the largest value	3

Here a = np.array([[1, 2], [3, 4]]). Pass axis=0 or axis=1 to compute down columns or across rows.

~4 min

Statistics Along an Axis

Choose which direction to summarise.

```
a = np.array([[1, 2], [3, 4]])
print(a.mean(axis=0))
print(a.sum(axis=1))
print(a.cumsum())
```

Output

```
[2. 3.]
[3 7]
[ 1 3 6 10]
```

Along an axis

axis=0 reduces down the columns. axis=1 reduces across the rows.

Cumulative methods

cumsum and cumprod keep every running total, instead of one final value.

~4 min

Boolean Array Methods

Count and test with true or false values.

```
a = np.array([4, -1, 7, 0])
print((a > 0).sum())
print((a > 0).any(), (a > 0).all())
print(a[a > 0])
```

Output

```
2
True False
[4 7]
```

True counts as 1

Summing a boolean array counts the True values, a quick way to count matches.

any and all

any is True if at least one item matches. all is True only if every item matches.

~4 min

Sorting

Order values, or find the order.

```
a = np.array([3, 1, 2])
print(np.sort(a))
b = np.array([[3, 1], [2, 5]])
print(np.sort(b, axis=1))
```

Output

```
[1 2 3]
[[1 3]
 [2 5]]
```

sort and argsort

np.sort returns a sorted copy. np.argsort returns the positions that would sort the array.

Along an axis

Pass axis= to sort each row or column separately. a.sort() sorts in place.

~4 min

Unique Values and Set Logic

Find distinct values and test membership.

```
a = np.array([3, 1, 3, 2, 1])
print(np.unique(a))
print(np.isin(a, [1, 2]))
```

Output

```
[1 2 3]
[False True False True True]
```

unique

Returns the sorted distinct values. Add return_counts=True to count each one.

Set logic

isin tests membership. union1d, intersect1d and setdiff1d combine two arrays as sets.

~4 min

Saving and Loading Arrays

Keep results between runs.

```
a = np.arange(4)
np.save("data.npy", a)
b = np.load("data.npy")
print(b)
```

Output

```
[0 1 2 3]
```

save and load

np.save stores one array in a binary .npy file. np.load brings it back with the same shape and dtype.

Text files

np.savetxt and np.loadtxt write and read plain text such as CSV.

~4 min

Random Numbers

Generate sample data with a seeded generator.

Call	What you get
rng.random(n)	n floats between 0 and 1
rng.integers(a, b, size)	Whole numbers from a up to b - 1
rng.normal(size=n)	Values from a bell curve
rng.choice(a, k)	A random sample of k items
rng.shuffle(a)	Reorders the array in place

Create rng with np.random.default_rng(42). A fixed seed makes the results repeatable.

~4 min

Linear Algebra

Matrix products and more.

```
a = np.array([[1, 2], [3, 4]])
print(a @ a)
print(np.trace(a))
```

Output

```
[[ 7 10]
 [15 22]]
5
```

Matrix product

@ (or np.dot) multiplies matrices. a * a multiplies element by element instead.

numpy.linalg

inv, det, solve and eig in np.linalg handle inverses, determinants and equation systems.

~3 min

A Typical Array Workflow

Each step works on whole arrays.

Step	Tool	Example
Load	np.loadtxt	Read numbers from a file
Clean	Boolean mask	a[a > 0]
Transform	Ufunc	np.sqrt(a)
Summarise	Statistics	a.mean(axis=0)
Save	np.save	Store the result

Because every step works on whole arrays, the code stays short and fast.

ONE-LINE TAKEAWAY

Process data by describing the result you want for the whole array, and let NumPy do the looping.

where mean sort unique

PYTHON FOR DATA · 35 MIN

File I/O With Arrays

.npy
One array
`np.save`

.npz
Many arrays
`np.savez`

Text
CSV and more
`np.savetxt`

Memmap
Huge files
`np.memmap`

~3 min

Four Ways to Store Array Data

Pick the format that fits the job.

1

Binary files

Exact copies of arrays, saved and loaded fast.

save savez

2

Text files

Readable CSV that other tools can open.

savetxt

3

Messy data

Files with gaps and missing values.

genfromtxt

4

Large files

Raw bytes and arrays too big for memory.

memmap

> Saving an array is easy. The skill is choosing a format you can read back correctly.

~4 min

Saving One Array

The .npy format keeps shape and dtype.

```
a = np.arange(6).reshape(2, 3)
np.save("a.npy", a)
b = np.load("a.npy")
print(b.shape, b.dtype)
print(np.array_equal(a, b))
```

Output

```
(2, 3) int64
True
```

.npy format

A binary file that stores one array with its shape and dtype, so it loads back exactly.

The extension

np.save adds .npy to the name if you leave it off.

~4 min

Saving Several Arrays

One archive, many named arrays.

```
x = np.arange(3)
y = np.ones(2)
np.savez("pack.npz", x=x, y=y)
d = np.load("pack.npz")
print(d.files)
print(d["y"])
```

Output

```
['x', 'y']
[1. 1.]
```

savez

Stores several arrays in one .npz archive, each under a keyword name.

Compressed

savez_compressed makes smaller files, at the cost of a little extra time.

~4 min

Choosing a Format

Match the format to who reads the file next.

Format	Save with	Best for
.npy	np.save	One array, fast and exact
.npz	np.savez	Several arrays together
Text or CSV	np.savetxt	Readable files and other tools
Raw binary	arr.tofile	Simple byte dumps, no shape stored
Memory map	np.memmap	Arrays larger than memory

Use .npy or .npz between Python runs, and text files when people or other tools must read the data.

~4 min

Writing Text Files

Save an array as CSV.

```
a = np.array([[1, 2.5], [3, 4]])
np.savetxt("a.csv", a,
           delimiter=",", fmt="%.1f")
print(open("a.csv").read())
```

Output

```
1.0,2.5
3.0,4.0
```

savetxt

Writes a 1-D or 2-D array as text. delimiter sets the separator, and fmt sets the number format.

Set fmt

The default format writes long scientific numbers, so choose a shorter fmt for tidy files.

~4 min

Reading Text Files

Turn CSV back into an array.

```
b = np.loadtxt("a.csv", delimiter=",")
print(b)
print(b.sum(axis=0))
```

Output

```
[[1. 2.5]
 [3. 4. ]]
[4. 6.5]
```

loadtxt

Reads numbers from a text file into an array. Every value becomes a float unless you set dtype.

Useful options

skiprows skips header lines, usecols picks columns, and comments= ignores notes after a symbol.

~4 min

Files With Missing Values

Gaps become nan instead of errors.

```
from io import StringIO
s = StringIO("1,2\n3,\n5,6")
a = np.genfromtxt(s, delimiter=",")
print(a)
print(np.isnan(a).sum())
```

Output

```
[[ 1. 2.]
 [ 3. nan]
 [ 5. 6.]]
1
```

genfromtxt

Handles missing values. Empty fields become nan, and filling_values can replace them.

loadtxt or genfromtxt

loadtxt is faster and strict. genfromtxt is slower but copes with gaps and mixed columns.

~4 min

Raw Binary Files

Bytes only, with no metadata.

```
a = np.array([1, 2, 3], dtype=np.int32)
a.tofile("raw.bin")
b = np.fromfile("raw.bin", dtype=np.int32)
print(b)
c = np.fromfile("raw.bin", dtype=np.int8)
print(c.size)
```

Output

```
[1 2 3]
12
```

No metadata

tofile writes only the bytes. Shape and dtype are not stored, so supply them when reading.

Same bytes, other view

Reading with a different dtype reinterprets the same bytes, so use the dtype you wrote.

~4 min

Memory-Mapped Arrays

Work with files larger than memory.

```
m = np.memmap("big.dat", dtype="float32",
              mode="w+", shape=(1000, 100))
m[0, :] = 1.5
m.flush()
print(m[0, :3])
print(m.shape)
```

Output

```
[1.5 1.5 1.5]
(1000, 100)
```

Memory mapping

A memmap acts like an array but reads from disk on demand, so huge files can be used.

flush and modes

flush() writes pending changes to the file. Modes: r reads, r+ edits, w+ creates.

~4 min

Using File Objects

Let a with block manage the file.

```
with open("a.npy", "wb") as f:
    np.save(f, np.arange(3))
with open("a.npy", "rb") as f:
    print(np.load(f))
```

Output

```
[0 1 2]
```

File objects work

np.save and np.load accept an open file, so a with block can close it for you.

Binary mode

Open .npy files with wb and rb. Text mode would corrupt the bytes.

np.load keeps allow_pickle off by default. Turn it on only for files you trust, because pickled data can run code.

~3 min

A Reading and Writing Checklist

Habits that prevent silent data errors.

Step	Habit	Reason
Pick a format	.npy for Python, CSV for sharing	Match the reader
State the dtype	Set dtype for raw and text files	Avoid wrong types
Check the shape	Print a.shape after loading	Catch misread columns
Close files	Use with blocks	No leaked file handles
Trust the source	Leave allow_pickle off	Avoid running unknown code

> Round-trip a small sample first: save it, load it, and compare it with the original.

ONE-LINE TAKEAWAY

Choose the file format first, then save, load and check, so the array comes back exactly as it went out.

save savez loadtxt memmap

PYTHON FOR DATA · 40 MIN

pandas Data Structures

Series
Labelled values
`pd.Series`

DataFrame
A table of columns
`pd.DataFrame`

Index
Labels that align
`df.index`

Selection
By label or position
`loc iloc`

~3 min

Four Building Blocks of pandas

Labelled data, built on top of NumPy arrays.

1

Series

One column of values with labels.

pd.Series

2

DataFrame

A table whose columns are Series.

pd.DataFrame

3

Index

The labels that line data up.

pd.Index

4

Selection

Pick rows and columns by label or position.

loc iloc

> pandas adds labels to NumPy arrays, so data lines up by name and not only by position.

~4 min

The Series

A 1-D array with an index of labels.

```
import pandas as pd
s = pd.Series([4, 7, 1], index=list("abc"))
print(s)
print(s["b"])
```

Output

```
a 4
b 7
c 1
dtype: int64
7
```

Values plus labels

A Series is a 1-D array with an index of labels. Its values are a NumPy array.

Label access

s['b'] looks up by label, and s.values gives the underlying NumPy array.

~4 min

Series From a Dictionary

Dictionary keys become the index.

```
s = pd.Series({"x": 1, "y": 2})
t = pd.Series([10, 20], index=["y", "z"])
print(s + t)
```

Output

```
x NaN
y 12.0
z NaN
dtype: float64
```

From a dict

Dictionary keys become the index, in the order the dictionary holds them.

Automatic alignment

Arithmetic matches items by label. A label found in only one Series gives NaN.

~4 min

The DataFrame

A table of columns that share one index.

```
df = pd.DataFrame({
    "name": ["Asha", "Ravi"],
    "age": [21, 24]})
print(df)
print(df.shape)
```

Output

```
   name age
0 Asha 21
1 Ravi 24
(2, 2)
```

A table of columns

A DataFrame is a table whose columns are Series that share one index.

Mixed dtypes

Each column has its own dtype, such as text, integers or dates.

~4 min

DataFrame Attributes

What a DataFrame can tell you about itself.

Attribute	Meaning	Example
df.shape	Rows and columns	(2, 2)
df.columns	Column labels	name, age
df.index	Row labels	0 and 1
df.dtypes	Type of each column	object, int64
df.values	The data as a NumPy array	A 2-D array

The examples refer to the name and age table. Call df.info() for a summary of columns, types and missing values.

~4 min

Selecting and Adding Columns

Work with whole columns at a time.

```
df = pd.DataFrame({"a": [1, 2, 3],
                   "b": [4, 5, 6]})
print(df["a"].tolist())
print(df[["a", "b"]].shape)
df["c"] = df["a"] + df["b"]
print(df["c"].tolist())
```

Output

```
[1, 2, 3]
(3, 2)
[5, 7, 9]
```

One column or several

df['a'] returns a Series. A list of names, df[['a', 'b']], returns a DataFrame.

Adding a column

Assign to a new name to create the column. Operations run on whole columns.

~4 min

Index Objects

The labels behind every Series and DataFrame.

```
idx = pd.Index(["a", "b", "c"])
print(idx[1])
print(idx.is_unique)
idx[1] = "z"
```

Output

```
b
True
TypeError
```

Immutable labels

An Index cannot be changed in place, so labels stay safe when shared between objects.

More than names

An index may repeat labels, and set-like methods such as union and intersection are built in.

~4 min

Reindexing

Conform data to a new set of labels.

```
s = pd.Series([1, 2], index=["a", "b"])
r = s.reindex(["b", "a", "c"])
print(r)
```

Output

```
b 2.0
a 1.0
c NaN
dtype: float64
```

reindex

Builds a new object that follows the new labels, adding NaN where a label had no data.

fill_value

Pass fill_value=0 to fill new labels with a value instead of NaN.

~4 min

Selecting With loc and iloc

Labels with loc, positions with iloc.

```
df = pd.DataFrame({"a": [10, 20, 30]},
                  index=["x", "y", "z"])
print(df.loc["y", "a"])
print(df.iloc[2, 0])
print(df.loc["x":"y"].shape)
```

Output

```
20
30
(2, 1)
```

loc uses labels

df.loc[row, col] selects by label. A label slice includes the end label.

iloc uses positions

df.iloc[row, col] selects by integer position. Its slices leave out the end, like Python lists.

~4 min

Dropping and Sorting

Remove labels, and reorder data.

```
s = pd.Series([3, 1, 2], index=list("cab"))
print(s.sort_index().index.tolist())
print(s.sort_values().tolist())
print(s.drop("a").tolist())
```

Output

```
['a', 'b', 'c']
[1, 2, 3]
[3, 2]
```

Sorting

sort_index orders by label, and sort_values orders by value.

drop

drop returns a new object without the labels you name. The original stays unchanged.

~3 min

Quick Looks at Data

First steps with any new table.

Method	What it shows
head()	The first few rows
info()	Columns, dtypes and missing values
describe()	Count, mean, std and quartiles
value_counts()	How often each distinct value appears
isna()	True wherever a value is missing

Start every new dataset with head, info and describe before any analysis.

ONE-LINE TAKEAWAY

A DataFrame is a table of Series, and the index is what lines every value up by label.

Series DataFrame Index loc / iloc

PYTHON FOR DATA · 45 MIN

pandas Essential Functionality

Reshape
Reindex and drop
`reindex`

Select
Filter and slice
`df[df.a > 2]`

Align
Arithmetic by label
`a + b`

Apply
Functions and ranks
`apply rank`

~3 min

Four Everyday Skills in pandas

The operations you will use on almost every dataset.

1

Reshape

Conform to new labels, and drop what you do not need.

reindex drop

2

Select

Filter rows by condition, and pick by label or position.

loc iloc

3

Align

Arithmetic that matches items by label.

add sub

4

Apply

Run functions, rank values and summarise.

apply rank

Each skill relies on the same idea: pandas lines data up by label.

~4 min

Reindexing With Fill Methods

Fill the gaps that new labels create.

```
s = pd.Series(["a", "b"], index=[0, 3])
print(s.reindex(range(5), method="ffill"))
```

Output

```
0 a
1 a
2 a
3 b
4 b
dtype: object
```

method='ffill'

Fills each new label from the previous value. It needs a sorted index.

Rows and columns

On a DataFrame, reindex takes the row labels first, or columns= for the column labels.

~4 min

Dropping Entries

Remove rows or columns by label.

```
import numpy as np
df = pd.DataFrame(np.arange(6).reshape(2, 3),
                  columns=["a", "b", "c"])
print(df.drop("b", axis=1).columns.tolist())
print(df.drop(0).shape)
```

Output

```
['a', 'c']
(1, 3)
```

Drop by label

Rows are dropped by default. Pass axis=1 or columns= to drop columns instead.

A new object

drop returns a new object, so assign the result if you want to keep it.

~4 min

Filtering Rows

Choose rows with a condition.

```
df = pd.DataFrame({"a": [1, 5, 9],
                   "b": [4, 2, 6]})
print(df[df["a"] > 2].index.tolist())
print(df[(df["a"] > 2) & (df["b"] > 3)].index.tolist())
```

Output

```
[1, 2]
[2]
```

Boolean filtering

A True/False Series picks the rows where it is True.

Combine with & and |

Wrap each condition in brackets. Use & for and, and | for or. The words and and or do not work on Series.

~4 min

Ways to Select Data

One table for the common forms.

Form	Example	Selects
Column	df['a']	One column, as a Series
Columns	df[['a', 'b']]	Several columns, as a DataFrame
By label	df.loc[r, c]	Rows and columns by label
By position	df.iloc[i, j]	Rows and columns by position
By condition	df[df['a'] > 2]	Rows where the test is True

Prefer loc and iloc when selecting rows and columns together, because their meaning is always clear.

~4 min

Integer Index Pitfalls

Is 0 a label or a position?

```
s = pd.Series([10, 20, 30], index=[2, 1, 0])
print(s[0])
print(s.iloc[0])
print(s.loc[0])
```

Output

```
30
10
30
```

Labels win

With an integer index, s[0] means the label 0, not the first item.

Be explicit

Use loc for labels and iloc for positions, so the code means the same thing for any index.

~4 min

Arithmetic and Alignment

Items are matched by label.

```
a = pd.Series([1, 2], index=["r", "s"])
b = pd.Series([10, 20], index=["s", "t"])
print((a + b).tolist())
print(a.add(b, fill_value=0).tolist())
```

Output

```
[nan, 22.0, nan]
[1.0, 22.0, 20.0]
```

Union of labels

The result has every label from both objects, with NaN where one side is missing.

add and fill_value

Methods such as add, sub, mul and div take fill_value, which stands in for missing items.

~4 min

DataFrame and Series Arithmetic

Match a Series to rows or columns.

```
df = pd.DataFrame([[1, 2], [3, 4]],
                  columns=["a", "b"])
row = df.iloc[0]
print((df - row).values.tolist())
print(df.sub(df.a, axis=0).values.tolist())
```

Output

```
[[0, 0], [2, 2]]
[[0, 1], [0, 1]]
```

Row-wise by default

A Series is matched to the columns, and applied to every row.

axis=0 for columns

To match a Series to the rows instead, use a method such as sub with axis=0.

~5 min

Applying Functions

Run a function over columns or cells.

```
df = pd.DataFrame({"a": [1, 4], "b": [9, 16]})
print(df.apply(np.sqrt).values.tolist())
print(df.apply(lambda c: c.max()).tolist())
print(df["a"].map(lambda v: v * 10).tolist())
```

Output

```
[[1.0, 3.0], [2.0, 4.0]]
[4, 16]
[10, 40]
```

apply

Runs a function on each column, or each row with axis=1, and combines the results.

map

Series.map applies a function to every item. DataFrame.map (applymap in older versions) does so for every cell.

~4 min

Sorting and Ranking

Order values, and rank them.

```
s = pd.Series([7, 1, 7, 3])
print(s.sort_values(ascending=False).tolist())
print(s.rank().tolist())
print(s.rank(method="first").tolist())
```

Output

```
[7, 7, 3, 1]
[3.5, 1.0, 3.5, 2.0]
[3.0, 1.0, 4.0, 2.0]
```

Ranking

rank gives each item its position in sorted order. Ties get the average rank by default.

Tie methods

method can be 'first', 'min', 'max' or 'dense' to choose another rule for tied values.

~4 min

Duplicate Labels

Index labels do not have to be unique.

```
s = pd.Series(range(3), index=["a", "a", "b"])
print(s.index.is_unique)
print(s["a"].tolist())
print(s["b"])
```

Output

```
False
[0, 1]
2
```

Duplicates allowed

Unlike dictionary keys, index labels can repeat. is_unique tells you whether they do.

Selection changes

A repeated label returns a Series, and a unique label returns a single value.

~3 min

Summarising Data

Common reductions on a DataFrame.

Method	What it gives
sum()	Total of each column
mean()	Average of each column
idxmax()	Label of the largest value in each column
count()	Number of non-missing values
describe()	Several statistics at once

Pass axis=1 to summarise each row, and skipna=False to include missing values.

ONE-LINE TAKEAWAY

In pandas, labels drive everything, so reindex, select, align and apply all work by matching names.

reindex filter align apply

PYTHON FOR DATA · 45 MIN

Summarizing and Computing Descriptive Statistics

Summaries
Totals and averages
`sum mean`

Spread
How values vary
`std quantile`

Counts
How often values occur
`value_counts`

Relationships
How columns move
`corr cov`

~3 min

Four Ways to Summarize Data

From a single total to the shape of a dataset.

1

Reductions

Collapse columns or rows to one value.

sum mean

2

Distribution

Describe spread, quartiles and extremes.

describe

3

Counts

Find distinct values and how often they occur.

value_counts

4

Relationships

Measure how two columns move together.

corr cov

A few well-chosen numbers can describe thousands of rows.

~4 min

Reductions

Collapse a DataFrame along an axis.

```
df = pd.DataFrame({"a": [1, 2, 3],
                   "b": [4, 5, 6]})
print(df.sum().tolist())
print(df.mean(axis=1).tolist())
print(df.idxmax().tolist())
```

Output

```
[6, 15]
[2.5, 3.5, 4.5]
[2, 2]
```

Down each column

Reductions work down each column by default. Pass axis=1 to work across each row.

idxmax and idxmin

idxmax returns the label of the largest value, and idxmin the label of the smallest.

~4 min

Missing Values in Summaries

NaN is skipped unless you say otherwise.

```
s = pd.Series([1.0, None, 3.0])
print(s.sum(), s.mean())
print(s.sum(skipna=False))
print(s.count())
```

Output

```
4.0 2.0
nan
2
```

Skipped by default

NaN is ignored, so the mean above is 4 divided by 2.

skipna=False

Turn skipping off and any NaN makes the whole result NaN. count tells you how many values are not missing.

~4 min

Common Summary Methods

Shown for the values 2, 4, 4 and 6.

Method	Meaning	Result
sum()	Total of the values	16
mean()	Average	4.0
median()	Middle value	4.0
var()	Variance	2.667
std()	Standard deviation	1.633

pandas var and std divide by n - 1 by default. NumPy divides by n, so the two can give different answers.

~4 min

Describing a Distribution

Quartiles and spread in one call.

```
s = pd.Series([2, 4, 4, 6])
print(s.describe().round(2).tolist())
print(s.quantile(0.5))
print(s.quantile([0.25, 0.75]).tolist())
```

Output

```
[4.0, 4.0, 1.63, 2.0, 3.5, 4.0, 4.5, 6.0]
4.0
[3.5, 4.5]
```

describe

Returns count, mean, std, min, the quartiles and max in one call.

quantile

quantile(q) gives the value below which a fraction q of the data falls.

~4 min

Mean or Median?

Outliers pull the mean away.

```
df = pd.DataFrame({"a": [1, 2, 3, 4],
                   "b": [10, 20, 30, 100]})
print(df.mean().tolist())
print(df.median().tolist())
```

Output

```
[2.5, 40.0]
[2.5, 25.0]
```

Outliers pull the mean

The 100 drags the mean of b up to 40, while its median stays at 25.

Report both

A large gap between mean and median suggests skewed data or outliers.

~4 min

Running Totals and Change

Keep a value for every row.

```
s = pd.Series([100, 110, 99])
print(s.cumsum().tolist())
print(s.diff().tolist())
print(s.pct_change().round(2).tolist())
```

Output

```
[100, 210, 309]
[nan, 10.0, -11.0]
[nan, 0.1, -0.1]
```

Running results

cumsum, cumprod, cummax and cummin keep a value for every row.

Change over time

diff gives the difference from the previous row, and pct_change gives the relative change.

~4 min

Unique Values and Counts

What occurs, and how often.

```
s = pd.Series(["a", "b", "a", "c", "a"])
print(s.unique().tolist())
print(s.value_counts().to_dict())
print(s.isin(["a", "c"]).sum())
```

Output

```
['a', 'b', 'c']
{'a': 3, 'b': 1, 'c': 1}
4
```

unique and nunique

unique lists the distinct values, and nunique counts them.

value_counts

Counts each distinct value, most common first. Add normalize=True for fractions.

~4 min

Correlation and Covariance

How two columns move together.

```
df = pd.DataFrame({"x": [1, 2, 3, 4],
                   "y": [2, 4, 6, 8],
                   "z": [4, 3, 2, 1]})
print(df.corr()["x"].tolist())
print(round(df["x"].cov(df["y"]), 2))
print(df["x"].corr(df["z"]))
```

Output

```
[1.0, 1.0, -1.0]
3.33
-1.0
```

Correlation

corr measures how strongly two columns move together, from -1 to 1.

Covariance

cov shows the same direction in the units of the data, so it is harder to compare.

~3 min

Reading a Correlation

What the number tells you.

Value	Meaning
1	Perfect rise together
0.7	Strong positive link
0	No straight-line link
-0.3	Weak negative link
-1	Perfect opposite movement

> Correlation shows that two columns move together. It does not show that one causes the other.

~3 min

Choosing the Right Summary

Start from the question you are asking.

Question	Use
What is typical?	mean() and median()
How spread out is it?	std() and quantile()
What is largest, and where?	max() and idxmax()
How many are there?	count() and value_counts()
Do two columns move together?	corr()

Check the median as well as the mean, because extreme values pull the mean.

ONE-LINE TAKEAWAY

Pick the summary that answers your question, and look at more than one number before you trust it.

mean describe value_counts corr

PYTHON FOR DATA · 40 MIN

Handling Missing Data

Detect
Find the gaps
`isna`

Drop
Remove them
`dropna`

Fill
Replace them
`fillna`

Estimate
Infer values
`interpolate`

~3 min

Four Responses to Missing Data

Real datasets always have gaps.

1

Detect

Find out where values are missing, and how many.

isna notna

2

Drop

Remove rows or columns with gaps.

dropna

3

Fill

Replace gaps with a constant or a statistic.

fillna

4

Estimate

Infer gap values from their neighbours.

interpolate

> Missing data is normal. What matters is handling it on purpose.

~4 min

What Missing Looks Like

pandas marks gaps with NaN.

```
import numpy as np
s = pd.Series([1, None, 3, np.nan])
print(s.tolist())
print(s.isna().tolist())
print(s.notna().sum())
```

Output

```
[1.0, nan, 3.0, nan]
[False, True, False, True]
2
```

NaN

A float value that pandas uses to mark missing numbers. None becomes NaN in numeric data.

isna and notna

isna is True where data is missing. notna is the opposite, and its sum counts the values that are present.

~3 min

Kinds of Missing Marker

Several markers, one meaning.

Marker	Where it appears	Note
NaN	Float columns	numpy.nan, the classic marker
None	Object columns	Python's own empty value
pd.NA	Nullable dtypes such as Int64	A newer, more general marker
NaT	Datetime columns	A missing timestamp

pandas treats all of these as missing, so isna and dropna handle each one the same way.

~4 min

Marking Missing Values

Turn placeholder values into NaN.

```
s = pd.Series([5, -999, 7])
print(s.replace(-999, np.nan).tolist())
print(s.mask(s < 0).tolist())
```

Output

```
[5.0, nan, 7.0]
[5.0, nan, 7.0]
```

Sentinel values

Data often uses -999 or 'N/A' to mean missing. Replace these with NaN so pandas can see them.

na_values when reading

read_csv(na_values=[...]) converts such markers while the file is loaded.

~4 min

Counting What Is Missing

Measure the gaps before acting.

```
df = pd.DataFrame({"a": [1, None, 3],
                   "b": [None, None, 6]})
print(df.isna().sum().tolist())
print(df.isna().sum().sum())
print(df.isna().mean().round(2).tolist())
```

Output

```
[1, 2]
3
[0.33, 0.67]
```

Count per column

isna().sum() counts the missing values in each column.

Share missing

isna().mean() gives the fraction missing, which helps you decide what to drop.

~4 min

Dropping Missing Data

Remove rows or columns with gaps.

```
df = pd.DataFrame({"a": [1, None, 3],
                   "b": [4, 5, None]})
print(df.dropna().shape)
print(df.dropna(how="all").shape)
print(df.dropna(axis=1).shape)
```

Output

```
(1, 2)
(3, 2)
(3, 0)
```

Rows by default

dropna removes every row that has at least one missing value.

how, axis and thresh

how='all' needs every value missing. axis=1 drops columns instead of rows.

~4 min

Controlling dropna

Decide which gaps matter.

```
df = pd.DataFrame({"a": [1, None, 3],
                   "b": [4, 5, None]})
print(df.dropna(subset=["a"]).index.tolist())
print(df.dropna(thresh=2).index.tolist())
```

Output

```
[0, 2]
[0]
```

subset

Look only at some columns when deciding which rows to drop.

thresh

Keep a row only if it has at least n values that are not missing.

~4 min

Filling Missing Data

Replace gaps with a chosen value.

```
s = pd.Series([1.0, None, 3.0])
print(s.fillna(0).tolist())
print(s.fillna(s.mean()).tolist())
df = pd.DataFrame({"a": [1, None],
                   "b": [None, 2]})
print(df.fillna({"a": 0, "b": 9}).values.tolist())
```

Output

```
[1.0, 0.0, 3.0]
[1.0, 2.0, 3.0]
[[1.0, 9.0], [0.0, 2.0]]
```

A constant or a statistic

fillna(0) uses one value everywhere. fillna(s.mean()) fills with the average.

Per-column values

Pass a dictionary to give each column its own fill value.

~4 min

Forward and Backward Fill

Carry real values across a gap.

```
s = pd.Series([1.0, None, None, 4.0])
print(s.ffill().tolist())
print(s.bfill().tolist())
print(s.ffill(limit=1).tolist())
```

Output

```
[1.0, 1.0, 1.0, 4.0]
[1.0, 4.0, 4.0, 4.0]
[1.0, 1.0, nan, 4.0]
```

Forward and backward

ffill carries the last real value forward. bfill pulls the next real value backward.

limit

limit=n stops after n filled values, so long gaps stay partly missing.

~4 min

Interpolation

Estimate values between known points.

```
s = pd.Series([1.0, None, None, 4.0])
print(s.interpolate().tolist())
t = pd.Series([1, None, 9], index=[0, 1, 4])
print(t.interpolate(method="index").tolist())
```

Output

```
[1.0, 2.0, 3.0, 4.0]
[1.0, 3.0, 9.0]
```

Linear by default

interpolate draws a straight line between the known points on each side of a gap.

method='index'

Spaces the filled values by the index, which suits uneven time or position labels.

~4 min

Choosing a Strategy

Match the fix to the situation.

Situation	Strategy	Tool
A few rows have gaps	Drop those rows	dropna()
A column is mostly empty	Drop the column	dropna(axis=1)
A sensible default exists	Fill with a constant	fillna(0)
Numeric measurements	Fill with mean or median	fillna(s.median())
Ordered data such as time	Carry forward or interpolate	ffill(), interpolate()

Note how many values you changed, because filling can hide a problem in the data.

ONE-LINE TAKEAWAY

Find the gaps first, then drop, fill or estimate on purpose, and record what you changed.

isna dropna fillna interpolate

PYTHON FOR DATA · 40 MIN

Hierarchical Indexing

MultiIndex
Several label levels
`from_product`

Selecting
Outer and inner labels
`s['a'] xs`

Reshaping
Long to wide
`unstack`

By level
Summaries per level
`groupby(level=0)`

~3 min

Four Skills With Multi-Level Labels

Rows identified by more than one label.

1

Build

Create an index with two or more levels.

MultiIndex

2

Select

Pick by outer label, inner label or both.

loc xs

3

Reorder

Swap and sort levels, or turn columns into levels.

swaplevel

4

Reshape

Move levels between rows and columns.

unstack

> A hierarchical index lets one table hold data about groups within groups.

~4 min

A Series With Two Levels

Each row has an outer and an inner label.

```
s = pd.Series([10, 20, 30],
              index=[["a", "a", "b"],
                     [1, 2, 1]])
print(s)
```

Output

```
a 1 10
  2 20
b 1 30
dtype: int64
```

Two levels of labels

A MultiIndex gives each row a tuple of labels such as ('a', 1), shown as grouped rows.

Ways to build one

Pass lists of labels, or use MultiIndex.from_tuples, from_arrays or from_product.

~3 min

Building a MultiIndex

Four common routes.

Constructor	Input	Result
from_tuples	A list of tuples	One row per tuple
from_arrays	One list of labels per level	Labels paired by position
from_product	Lists of labels	Every combination of them
set_index	Columns of a DataFrame	Those columns become levels

from_product builds a full grid, so 2 outer labels and 3 inner labels give 6 rows.

~4 min

Partial Indexing

Select by outer or inner label.

```
s = pd.Series([10, 20, 30, 40],
              index=[["a", "a", "b", "b"],
                     [1, 2, 1, 2]])
print(s["a"].tolist())
print(s["b", 2])
print(s.loc[:, 1].tolist())
```

Output

```
[10, 20]
40
[10, 30]
```

Outer level first

s['a'] selects every row under the outer label a, and drops that level.

Inner levels

s['b', 2] picks one item. s.loc[:, 1] selects inner label 1 under every outer label.

~4 min

Cross-Sections With xs

Select on any level of a DataFrame.

```
idx = pd.MultiIndex.from_product(
    [["x", "y"], [1, 2]])
df = pd.DataFrame({"v": [1, 2, 3, 4]},
                  index=idx)
print(df.loc["x"]["v"].tolist())
print(df.xs(2, level=1)["v"].tolist())
```

Output

```
[1, 2]
[2, 4]
```

xs

xs(key, level=n) picks rows by a label at any level, not only the outer one.

Named levels

Name levels with names=[...] so you can refer to them by name instead of by number.

~4 min

Reordering and Sorting Levels

Change which level comes first.

```
s = pd.Series([1, 2, 3],
              index=[["b", "a", "b"],
                     [1, 1, 2]])
t = s.swaplevel().sort_index()
print(t.index.tolist())
print(s.sort_index().index.tolist())
```

Output

```
[(1, 'a'), (1, 'b'), (2, 'b')]
[('a', 1), ('b', 1), ('b', 2)]
```

swaplevel

Exchanges two levels, so an inner level can become the outer one.

Sort for speed

Selecting and slicing work best on a sorted index, so call sort_index after reordering levels.

~4 min

Columns to Index Levels

Move between columns and levels.

```
df = pd.DataFrame({"g": ["a", "a", "b"],
                   "n": [1, 2, 1],
                   "v": [10, 20, 30]})
h = df.set_index(["g", "n"])
print(h.index.nlevels, h.shape)
print(h.reset_index().columns.tolist())
```

Output

```
2 (3, 1)
['g', 'n', 'v']
```

set_index

Turns one or more columns into index levels. Those columns leave the table.

reset_index

Moves the levels back into ordinary columns, with a fresh 0, 1, 2 index.

~4 min

Unstacking and Stacking

Reshape between long and wide.

```
s = pd.Series([1, 2, 3, 4],
              index=[["a", "a", "b", "b"],
                     ["x", "y", "x", "y"]])
u = s.unstack()
print(u.values.tolist())
print(u.stack().shape)
```

Output

```
[[1, 2], [3, 4]]
(4,)
```

unstack

Pivots the innermost index level into columns, turning a long table into a wide one.

stack

The reverse. It pivots columns back into the innermost index level.

~4 min

Unstacking With Gaps

Missing combinations and other levels.

```
s = pd.Series([1, 2, 3],
              index=[["a", "a", "b"],
                     ["x", "y", "x"]])
print(s.unstack().values.tolist())
print(s.unstack(fill_value=0).values.tolist())
print(s.unstack(0).index.tolist())
```

Output

```
[[1.0, 2.0], [3.0, nan]]
[[1, 2], [3, 0]]
['x', 'y']
```

Missing combinations

unstack creates NaN wherever a pair of labels has no data.

Choose the level

unstack(0) pivots the outer level instead of the innermost. fill_value replaces the NaN.

~4 min

Summaries by Level

Reduce data over one index level.

```
s = pd.Series([1, 2, 3, 4],
              index=[["a", "a", "b", "b"],
                     [1, 2, 1, 2]])
print(s.groupby(level=0).sum().to_dict())
print(s.groupby(level=1).mean().to_dict())
```

Output

```
{'a': 3, 'b': 7}
{1: 2.0, 2: 3.0}
```

groupby on a level

groupby(level=n) groups rows by one index level, and a reduction summarises each group.

Not sum(level=)

Older pandas allowed sum(level=0). Current pandas needs groupby(level=0) instead.

~3 min

Hierarchical Indexing Cheat Sheet

Match the task to the tool.

Task	Tool
Build levels	from_product, set_index
Select by outer label	s['a']
Select on any level	xs(key, level=n)
Reorder levels	swaplevel, sort_index
Reshape	stack, unstack

> Name your levels. Names make code easier to read than the numbers 0 and 1.

ONE-LINE TAKEAWAY

A hierarchical index gives rows several labels, and stack and unstack move them between rows and columns.

MultiIndex · xs · swaplevel · unstack